Final Smart Contract Audit Report

HackChain - HACK Token and Ecosystem Mechanisms

Julian Ruiz - Blockchain Engineer / Smart Contract Security Reviewer

July 11, 2026

Contents

1 Executive Opinion **2**
1.1 Conformance Summary for the 30 Mechanisms . . . 3
1.2 Highest-Priority Risks . . . 3
1.3 Engineering Criteria Reviewed . . . 3

2 Scope and Sources **4**
2.1 Audited Code . . . 4
2.2 Business Documentation Included . . . 4
2.3 Out of Scope . . . 4

3 Methodology and Evidence **5**

4 Architecture and Trust Model **5**
4.1 Protected Assets . . . 5
4.2 Privileged Actors . . . 5
4.3 Minimum Required Invariants . . . 5

5 Mechanism-by-Mechanism Conformance Matrix **6**

6 Technical Findings Index **8**

7 High Findings **10**
7.1 H-01 - Referral Rewards Can Be Collected Immediately and Exploited Through Sybil Identities 10
7.2 H-02 - Staking Principal Repayment Is Conditioned on Reward Payment . . . 11
7.3 H-03 - Unlimited Academic Views Can Capture the Educator Pool . . . 11
7.4 H-04 - Penalties Depend on the Offender's Voluntary Allowance . . . 12
7.5 H-05 - Penalty Base Uses Movable Balances and Mass-Sale Checks Use Post-Sale State . . . 13

8 Medium Findings **14**
8.1 M-01 - `deposit()` Inflates Accounting Without Transferring Tokens . . . 14
8.2 M-02 - Immediate Fee Exemption and Stale Eligibility . . . 14
8.3 M-03 - Reputation Claim Fails Across Epoch Rollover . . . 14
8.4 M-04 - Off-Chain Events Lack Unique IDs and Idempotency . . . 15
8.5 M-05 - `Ownable` and `AccessControl` Administration Diverge . . . 15
8.6 M-06 - Recruiter Bonuses Do Not Prove Activity, Registration, or Coherent KYC . . . 15

9 Low and Informational Findings **16**
9.1 L-01 - Unsafe ERC-20 Transfer Handling . . . 16
9.2 L-02 - Stake Index Is Validated Too Late . . . 16
9.3 L-03 - Expired Advanced Membership Retains an Active Flag . . . 16
9.4 L-04 - Unbounded Loops and Batches . . . 16
9.5 L-05 - Project Sponsor Can Be Overwritten and Has No Refund Path . . . 16
9.6 I-01 - Thirty-Day Epochs Are Not Calendar Months . . . 16
9.7 I-02 - Eligible Rewards Are Not Reserved . . . 16

9.8 I-03 - Deployment and Role Configuration Are Not Reproducible from the Received Repository 16
9.9 I-04 - Code Quality and Observability 16
9.10 L-06 - Emergency Pausing Is Not Granular and Can Block Withdrawals 17
9.11 I-05 - Public Mappings and Duplicate Getters 17
9.12 I-06 - NatSpec Is Incomplete and Partially Misaligned with Behavior 18
9.13 I-07 - Conditional Modernization to `ReentrancyGuardTransient` 18
9.14 Reviewed Criterion without a Finding - `Contract__Error()` Convention 19

10 Business-Conformance Gaps **19**
10.1 BL-01 - Allocation, Vesting, and Liquidity Lock Are Not Enforced On-Chain (High) 19
10.2 BL-02 - The 5% Platform Transaction Fee Is Absent (High) 19
10.3 BL-03 - Personalized Classes Are Not Implemented (Medium) 20
10.4 BL-04 - Advanced Membership Is Incomplete (Medium) 20
10.5 BL-05 - Academic Membership Cancellation Is Missing and Sources Conflict (Medium) 20
10.6 BL-06 - Open-Source Project Funding Does Not Validate Roles, Repository, or Talents (Medium) 21
10.7 BL-07 - Impersonation and Plagiarism Do Not Follow the Documented Process (Medium) . . 21
10.8 BL-08 - Business Facts Rely on Trusted Off-Chain Calls without Attestations (Medium) 21
10.9 BL-09 - DAO/Governance Is Documented but Is Outside the Repository (Informational) . . . 21
10.10 BL-10 - Documentary Contradictions and Regulatory Exposure (Medium) 21

11 Recommended Remediation Architecture **22**
11.1 Layer 1 - Token and Treasury 22
11.2 Layer 2 - Business Evidence 22
11.3 Layer 3 - Payments and Rewards 22
11.4 Layer 4 - Penalties 22

12 Prioritized Remediation Plan **22**
12.1 Blocking before a Public Testnet 22
12.2 Before Mainnet 22

13 Minimum Post-Remediation Acceptance Tests **22**

14 Package Reproduction Guide **23**

15 Limitations **23**

16 Final Conclusion **23**

17 Appendix A - Evidence and Technical Environment **24**

18 Appendix B - Business-Document Hashes **24**

19 Appendix C - Technical References **24**

20 Appendix D - Severity Definitions **24**

1 Executive Opinion

Auditor: Julian Ruiz
Role: Blockchain Engineer - Smart Contract Security Reviewer
LinkedIn: linkedin.com/in/julian-ruiz-v
Result: NOT READY FOR PRODUCTION in the audited state.

The repository implements a significant portion of the proposed mechanisms, but it does not safely enforce several core economic conditions. Confirmed issues allow rewards to be collected before requirements are satisfied, paid activity to be fabricated, staking principal withdrawals to be blocked, and penalties to be avoided. The business-model review also identified missing mechanisms, documented rules that are not enforced in code, and contradictions that must be resolved before the specification can be considered final.

Technical severity	Count
Critical	0
High	5
Medium	6
Low	6
Informational	7
Total	**24**

In addition to the 24 technical findings and engineering observations, the report records **10 business-conformance gaps**. They are presented separately to avoid assigning duplicate severity when the same root cause affects multiple mechanisms.

1.1 Conformance Summary for the 30 Mechanisms

Status	Count	Interpretation
Fully compliant	0	No mechanism was found to be complete, secure, and self-contained against the complete specification.
Mostly implemented	3	The core function exists, but still depends on privileged/off-chain validation or requires hardening.
Partially implemented	25	An approximation exists, but conditions, flows, controls, or economic distribution are missing.
Absent	2	No equivalent contract or flow exists in the audited scope.

The two absent mechanisms are the **5% HACK transaction fee** and the **personalized-class payment/escrow flow with a 1,000 HACK minimum**.

1.2 Highest-Priority Risks

1. **Penalties are not enforceable:** the offender may withhold or revoke allowance, move tokens, or place them in staking to reduce the calculation base.
2. **Rewards are manipulable:** referral rewards can be collected immediately and academic views can be self-generated without limit.
3. **User funds can be locked:** repayment of staking principal depends on the incentives pool being able to pay the reward.
4. **Tokenomics are not enforced on-chain:** distribution, vesting, advisor allocation, and liquidity locking are not protected by contracts in scope.
5. **The specification is inconsistent:** diagrams, tables, NatSpec, and the business plan sometimes define different outcomes for the same action.

1.3 Engineering Criteria Reviewed

Reviewed criterion	Decision	Treatment in this report
Errors named as `Contract__Error()`	Not mandatory	Optional convention. It is not classified as a vulnerability and a style-only mass rename is not recommended.
`public mapping` plus equivalent explicit getters	Applicable	I-05 added for duplicate ABI surface, bytecode, and storage/interface coupling.
Improve NatSpec	Applicable	I-06 added for incomplete coverage and comments that do not always match behavior.
Migrate to `ReentrancyGuardTransient`	Conditionally applicable	I-07 added. Recommended when the network and toolchain support EIP-1153; it does not by itself remediate a demonstrated exploit.
Emergency stop preventing new deposits	Applicable	L-06 added. The existing token-wide pause is too broad and may also block withdrawals; granular inflow pausing is recommended.

These criteria do not change the five High or six Medium risks. They are classified as one Low finding and three Informational observations.

2 Scope and Sources

2.1 Audited Code

- Repository: Hackchain HACK Token Smart Contracts.
- Declared branch: `main`.
- Frozen commit: `290377e9862aeabaa2814bcd0bac9ddad2707ce8`.
- Solidity: `0.8.24`.
- OpenZeppelin used to reproduce the audited commit: `5.0.2`.
- OpenZeppelin used for a separate modernization compatibility check: `5.6.1`.
- Contracts: 11.
- Approximate Solidity lines: 3,052.
- Documented target network: Polygon.

2.2 Business Documentation Included

- **HACK Token Mechanisms**, 12 pages: tabular definition of 30 mechanisms.
- **Business Plan**, 11 pages: allocation, vesting, memberships, fees, projections, and investor returns.
- **HackChain Whitepaper 1.1.0**, 19 pages: architecture, value proposition, tokenomics, governance, roadmap, and business model.
- **Mechanism Diagrams**, one consolidated sheet: decision flows for all mechanisms.
- **Get to Know HackChain**, 12 pages: commercial presentation and ecosystem summary.

The documents were used as evidence of intended behavior. When sources disagree, this report does not silently select one interpretation; it records the contradiction and requires a formal product decision.

2.3 Out of Scope

- Backend, frontend, database, WalletConnect, Pinata, OpenSea, Alchemy, KYC, and identity services.
- ERC-721 certificate contracts, tokenized identity, DAO, vesting, sale, or liquidity contracts not present in the received repository.

- Production deployment addresses, multisig configuration, role-grant scripts, and real operational policies.
- Legal, tax, or regulatory review. Financial-document observations require specialized counsel and do not constitute legal advice.

3 Methodology and Evidence

1. Commit freeze and SHA-256 hashes.
2. Manual contract-by-contract review and cross-contract token-flow review.
3. Modeling of roles, assets, privileged state, and external dependencies.
4. Exact compilation with `solc-js 0.8.24`.
5. Slither 0.11.5 and manual triage of 69 raw signals.
6. Mocha, Ethers, and Ganache: **16 proof-of-concept tests passed**.
7. Foundry Forge 1.7.1: 7 passing tests, including 256-run fuzzing.
8. Halmos 0.3.3 + Z3: symbolic counterexample for the pool-solvency invariant.
9. Second verification against the business documents.
10. Review of custom-error conventions, getters, NatSpec, emergency pausing, and `ReentrancyGuardTransient` compatibility with OpenZeppelin 5.6.1, Solidity 0.8.24, and `evmVersion=cancun`.

Primary reproduction command:

```
npm ci
npm run compile
npm run test:js
```

Observed result:

```
Hackchain security proof-of-concepts: 16 passing
Hackchain business-logic conformance proof-of-concepts: 5 passing
Total: 16 passing (8s)
```

Ganache's `uWS` warning only enables a slower JavaScript fallback; it does not change the functional results.

4 Architecture and Trust Model

4.1 Protected Assets

- HACK maximum supply and allocation.
- Principal deposited in staking.
- Real and accounting balance of the incentives pool.
- Membership payments and the portions assigned to treasury/educators.
- Referral, event, reputation, education, and hiring rewards.
- Penalties and profile blocking.
- Private funds used to finance open-source projects.

4.2 Privileged Actors

Most business facts are registered through `ADMIN_ROLE` or `ENFORCER_ROLE`. The system is therefore not fully trustless: security depends on those roles remaining uncompromised and on the backend not replaying the same event. A multisig reduces single-key risk, but **does not replace** idempotency, signed attestations, or on-chain economic limits.

4.3 Minimum Required Invariants

- A user can always recover matured principal even when rewards are underfunded.
- `poolBalance` can never exceed the actual ERC-20 balance unless an explicit, traceable debt exists.
- An external business fact can only be consumed once.
- Penalties cannot be avoided by moving tokens after the offense.

- A paid view must correspond to a unique viewer, content item, and period.
- The sum of minted/allocated tokens by category must respect the cap and approved allocation.

5 Mechanism-by-Mechanism Conformance Matrix

#	Documented mechanism	Observed implementation	Status	Main gap
1	Staking: 1,000/30 days -> 50; 10,000/365 days -> 1,000	`Staking`	Partial	Principal and reward are coupled; no guaranteed reserve exists.
2	Referral: 1,000; max 5/month; referred user holds 1,000 for one month	`Referral`	Partial	Can be validated immediately and with Sybil wallets.
3	Advanced membership: 50k/140k/550k; 50/50; 1k cancellation	`Membership`	Partial	Only monthly tier exists; no refund; expired state is inconsistent.
4	Advanced-degree reward: 50k and tokenized-identity link	`Talent`	Partial	Degree level and tokenized identity are not validated on-chain.
5	Promotional event with >=10 attendees: 30k to organizer	`Events`	Mostly	`eventId` prevents double payment, but attendance and organizer depend on the enforcer.
6	No fee after 100k staked for >=12 months	`Staking`	Partial	Activates immediately and may remain after the annual stake is removed.
7	Identity fraud: 10% plus block/removal/correction	`Penalties`	Partial	Profile is not blocked and collection is avoidable.
8	5% HACK transaction fee to treasury	None	Absent	No router, settlement, or use of `hasNoCommission`.
9	Mass sale: >50% by holder with >=1% of circulating supply	`Penalties`	Partial	Uses post-sale balance and enforcer-supplied supply; amount basis conflicts across documents.
10	Talent attends 4 events in the month: 4k	`Events`	Partial	No unique event IDs; months are global 30-day epochs.

#	Documented mechanism	Observed implementation	Status	Main gap
11	Talent hired: 5k, once/month	`Talent`	Partial	No unique hiring ID or proof of platform registration.
12	Personalized class payment, minimum 1k/hour	None	Absent	Escrow, confirmation, dispute, and settlement are missing.
13	Academic membership 30k/80k/330k; 50/50	`Membership`	Partial	Prices and split exist; cancella-tion/refund flow does not.
14	Interview no-show: 5%; minimum 15-minute presence	`Penalties`	Partial	No `interviewId`, 15-minute proof, or snapshot; penalty is avoidable.
15	Best-reputation Talent: 5k, claim days 1-3	`Reputation`	Partial	Epoch rollover can invalidate an open claim window.
16	Best-reputation Educator: 5k, claim days 1-3	`Reputation`	Partial	Same epoch issue; voting/tie resolution is off-chain.
17	Best-reputation Recruiter: 5k, claim days 1-3	`Reputation`	Partial	Same epoch issue; voting/tie resolution is off-chain.
18	First finalized academic event with certificates: 5k	`Educator`	Partial	Finalization and certificate issuance are not proven on-chain.
19	Four finalized events in the same month: 4k	`Educator`	Partial	No `eventId`; exposed to replay and 30-day epoch ambiguity.
20	Implement official API: 50k	`Educator`	Partial	Fully privileged validation with no attestation or versioning.
21	Tokenize first 10 PDFs for 10 Talents: 10k	`Educator`	Mostly	IDs/counters exist, but certificate validity and recipient are off-chain.
22	First 10 active Talents trained: 10k	`Educator`	Mostly	Uniqueness exists; activity remains an enforcer assertion.

#	Documented mechanism	Observed implementation	Status	Main gap
23	Finance GitHub OSS by Educator/Recruiter: 1k/10k/100k	`Talent`	Partial	Any wallet and zero `projectId` accepted; no refund or role verification.
24	Educator bonus when trained Talent is hired: 5k	`Educator`	Partial	No `hiringId`; statistical selection is not encoded.
25	Educator inactive for 2 months: 5% plus block	`Penalties`	Partial	Detection off-chain; movable balance and voluntary allowance allow evasion.
26	Plagiarism: 10%, block, and payment to affected party	`Penalties`	Partial	External case goes to treasury; no mandatory atomic payment to the affected party.
27	Recruiter active for 7 days: 50k; one identity/wallet	`Recruiter`	Partial	Only elapsed time is measured; no activity or unique identity.
28	Hire 4 active Talents/month: 40k	`Recruiter`	Partial	No `hiringId` or activity proof.
29	Official KYC: 200k	`Recruiter`	Partial	Registration and unique KYC attestation are not required.
30	Recruiter has no interviews for 2 months: 5% plus block	`Penalties`	Partial	Detection off-chain; balance/allowance permit evasion.

6 Technical Findings Index

ID	Severity	Finding	Contracts
H-01	High	Referral rewards can be collected immediately and exploited through Sybil identities	ReferralSystem, StakingContract
H-02	High	Staking principal repayment is conditioned on reward payment	StakingContract, IncentivesPool
H-03	High	Unlimited academic views can capture the educator pool	MembershipSystem

ID	Severity	Finding	Contracts
H-04	High	Penalties depend on the offender’s voluntary allowance	PenaltySystem, HackToken
H-05	High	Penalty base uses movable balances and mass-sale checks use post-sale state	PenaltySystem, StakingContract
M-01	Medium	`deposit()` inflates accounting without transferring tokens	IncentivesPool
M-02	Medium	Immediate fee exemption and stale eligibility	StakingContract
M-03	Medium	Reputation claim fails across epoch rollover	ReputationBunises
M-04	Medium	Off-chain events lack unique IDs and idempotency	Multiple
M-05	Medium	`Ownable` and `AccessControl` administration diverge	HackToken
M-06	Medium	Recruiter bonuses do not prove activity, registration, or coherent KYC	RecruiterBonuses
L-01	Low	ERC-20 transfers do not use `SafeERC20` consistently	MembershipSystem, PenaltySystem
L-02	Low	Stake index is used before validation	StakingContract
L-03	Low	Expired advanced membership keeps `active=true`	MembershipSystem
L-04	Low	Unbounded loops/batches may exceed block gas	StakingContract, TalentBonuses
L-05	Low	Project sponsor can be overwritten and refunds are absent	TalentBonuses
L-06	Low	Emergency pausing is not granular and can block withdrawals	HackToken, StakingContract, MembershipSystem, IncentivesPool
I-01	Info	“Month” is modeled as a global 30-day epoch	Multiple
I-02	Info	Rewards are not reserved and depend on future liquidity	Multiple
I-03	Info	Reproducible deployment, CI, and role configuration are missing	Repository
I-04	Info	Technical debt, dead state, and incomplete observability	Multiple

ID	Severity	Finding	Contracts
I-05	Info	Public mappings and duplicate getters expand ABI and couple storage	Multiple
I-06	Info	NatSpec is incomplete and partly misaligned with logic	Multiple
I-07	Info	Conditional modernization to current OpenZeppelin and `ReentrancyGuard-Transient`	Multiple

7 High Findings

7.1 H-01 - Referral Rewards Can Be Collected Immediately and Exploited Through Sybil Identities

Business requirement. The referred user must keep at least 1,000 HACK staked for one month. The referrer receives 1,000 HACK and is limited to five referrals per month.

Root cause. `validateReferral()` checks only the currently deposited aggregate. It does not verify the age or exact duration of a specific position, nor does it bind the referral registration to a stake or unique identity.

Local reproduction flow.

1. The attacker controls wallet A (referrer) and wallet B (referred user).
2. B stakes 1,000 HACK for 30 days.
3. B registers A as referrer.
4. The referral is validated in the same block or immediately afterward.
5. A receives 1,000 HACK although one month has not elapsed.
6. The attacker repeats the process with A2, A3, and other identities to distribute the per-address limit.

Impact. Reward inflation and incentives-pool depletion without real user acquisition or retention.

Recommended remediation. Bind every referral to a `stakeId`, register the association before eligibility, consume a unique `referralId`, and require actual maturity. Sybil resistance requires identity/KYC or an off-chain proof-of-humanity score; a wallet-only limit is insufficient.

```
mapping(bytes32 => bool) public referralConsumed;

function validateReferral(
    bytes32 referralId,
    address referrer,
    address referred,
    uint256 stakeId
) external onlyRole(VALIDATOR_ROLE) {
    require(!referralConsumed[referralId], "referral used");
    require(referrer != referred, "self referral");

    Stake memory s = staking.getStake(referred, stakeId);
    require(s.amount >= 1_000 ether, "insufficient stake");
    require(s.duration >= 30 days, "wrong duration");
    require(block.timestamp >= s.startTime + 30 days, "not matured");
    require(s.active, "stake inactive");

    referralConsumed[referralId] = true;
```

```
    incentivesPool.distribute(referrer, 1_000 ether, "referral");
}
```

Evidence: `HC-H01` and `test_ReferralRewardCanBeClaimedImmediatelyAfterStake`.

7.2 H-02 - Staking Principal Repayment Is Conditioned on Reward Payment

Root cause. `unstake()` returns principal and pays the reward in one atomic transaction. If `IncentivesPool.distribute()` reverts because of insolvency, pausing, or missing permissions, the EVM also reverts the principal transfer.

Local reproduction flow.

1. A user stakes 1,000 HACK.
2. Time advances by 30 days.
3. The incentives pool has no funds or staking no longer has `DISTRIBUTOR_ROLE`.
4. The user calls `unstake(stakeId)`.
5. Reward distribution reverts.
6. The entire transaction reverts and the user's principal remains locked.

Impact. All matured positions may be frozen by an external economic or operational dependency.

Recommended remediation. Separate principal and reward. Underfunding must only affect the reward. Record `pendingRewards`, allow later claims, and provide emergency principal withdrawal.

```
function unstake(uint256 stakeId) external nonReentrant {
    Stake storage s = _stake(msg.sender, stakeId);
    require(block.timestamp >= s.unlockAt, "locked");
    require(s.active, "inactive");

    s.active = false;
    uint256 principal = s.amount;
    uint256 reward = _rewardFor(s);

    hackToken.safeTransfer(msg.sender, principal);
    pendingRewards[msg.sender] += reward;
    emit PrincipalWithdrawn(msg.sender, stakeId, principal);
}

function claimReward(uint256 amount) external nonReentrant {
    require(pendingRewards[msg.sender] >= amount, "exceeds debt");
    pendingRewards[msg.sender] -= amount;
    incentivesPool.distribute(msg.sender, amount, "staking_reward");
}
```

Evidence: `HC-H02` and `test_EmptyRewardPoolBlocksPrincipalWithdrawal`.

7.3 H-03 - Unlimited Academic Views Can Capture the Educator Pool

Root cause. Any active academic member may call `registerContentView(educator)` without limit. There is no `contentId`, `viewId`, unique viewer, period limit, or self-view restriction.

Local reproduction flow.

1. The attacker obtains `EDUCATOR_ROLE` and purchases an academic membership.
2. `pendingEducatorPool` contains funds contributed by other memberships.
3. The attacker calls `registerContentView(attacker)` hundreds of times.
4. The attacker's views represent 100% of the counter.
5. The attacker calls `claimEducatorRewards()` and receives the complete pool.

Impact. Misappropriation of revenue intended for legitimate educators and failure of the consumption-based distribution model.

Recommended remediation. Register paid views through an EIP-712 attestation from an authorized service, including `viewId`, viewer, educator, content, cycle, expiration, and nonce. Block self-views and close/snapshot each cycle before distribution.

```
mapping(bytes32 => bool) public usedView;

function registerContentView(View calldata v, bytes calldata sig) external {
    require(v.viewer != v.educator, "self view");
    require(v.expiresAt >= block.timestamp, "expired");
    bytes32 viewId = keccak256(
        abi.encode(v.viewer, v.educator, v.contentId, v.cycleId)
    );
    require(!usedView[viewId], "duplicate view");
    require(_verifyViewSigner(v, sig), "invalid attestation");

    usedView[viewId] = true;
    views[v.cycleId][v.educator] += 1;
    totalViews[v.cycleId] += 1;
}
```

Evidence: `HC-H04` and `test_EducatorCanSelfGenerateViewsAndDrainEducatorPool`.

7.4 H-04 - Penalties Depend on the Offender's Voluntary Allowance

Root cause. `_applyPenalty()` collects through `transferFrom(user, destination, amount)`. The offender can withhold or revoke approval. Because profile blocking is recorded after the transfer, the block also reverts.

Local reproduction flow.

1. The penalizable wallet holds HACK but has not approved `PenaltySystem`.
2. The enforcer calls a penalty function.
3. `transferFrom()` reverts because allowance is insufficient.
4. No tokens move and `profileBlocked` remains unchanged.

Impact. Disciplinary mechanisms are optional for the offender.

Possible remediations.

- **Debt model:** block the profile first, record debt, and require payment before unblocking. This is the least invasive model.
- **Escrow/collateral:** require a deposited balance before access to services subject to penalties.
- **Limited token hook:** grant an auditable penalty contract tightly bounded burn/transfer powers, protected by multisig, limits, and timelock. This is more coercive and increases centralization risk.

```
function imposePenalty(address user, bytes32 caseId, uint256 amount)
    external
    onlyRole(ENFORCER_ROLE)
{
    require(!cases[caseId].exists, "case used");
    cases[caseId] = Case({
        user: user,
        debt: amount,
        exists: true,
        paid: false
    });
    profileBlocked[user] = true;
    emit PenaltyDebtCreated(caseId, user, amount);
}

function payPenalty(bytes32 caseId) external nonReentrant {
    Case storage c = cases[caseId];
```

```
    require(msg.sender == c.user && !c.paid, "invalid case");
    hackToken.safeTransferFrom(msg.sender, incentivesPool, c.debt);
    c.paid = true;
    profileBlocked[msg.sender] = false;
}
```

Evidence: `HC-H05` and `test_PenaltyCanBeAvoidedByWithholdingAllowance`.

7.5 H-05 - Penalty Base Uses Movable Balances and Mass-Sale Checks Use Post-Sale State

Root cause. Percentage penalties read `balanceOf(user)` at enforcement time. The user can move or stake the balance before the enforcer acts. For a mass sale, the contract compares `saleAmount` and the 1% threshold against the **post-sale balance**, while accepting `circulatingSupply` supplied by the enforcer.

Flow A - reduce the base to zero through staking.

1. The user owns 100,000 HACK.
2. Before enforcement, the user deposits all 100,000 HACK into `StakingContract`.
3. `PenaltySystem.balanceOf(user)` returns zero.
4. The calculated penalty is zero and reverts with `AmountMustBeGreaterThanZero`.

Flow B - make a mass sale undetectable.

1. The user owns 2% of circulating supply.
2. The user sells enough HACK to fall below 1%.
3. The enforcer calls `applyMassSalePenalty` after the sale.
4. The contract evaluates the new balance and concludes the user did not own 1%.
5. The penalty cannot be applied although the pre-sale condition was satisfied.

Impact. General penalty evasion and false negatives in anti-whale controls.

Recommended remediation. Define an offense attestation with verifiable snapshot fields: `preBalance`, `saleAmount`, `circulatingSupply`, transaction hash, and block. Verify it through a signed oracle attestation or integrate the fee into the actual sale/settlement path. Platform penalties should create debt from the snapshot rather than current balance.

```
struct MassSaleAttestation {
    address user;
    uint256 preBalance;
    uint256 saleAmount;
    uint256 circulatingSupply;
    uint256 observedBlock;
    bytes32 txHash;
    uint256 nonce;
    uint256 deadline;
}

function registerMassSale(
    MassSaleAttestation calldata a,
    bytes calldata sig
) external {
    require(_verifyOracle(a, sig), "invalid proof");
    require(a.preBalance * 100 >= a.circulatingSupply, "below 1 percent");
    require(a.saleAmount * 100 > a.preBalance * 50, "below 50 percent");
    uint256 debt = a.saleAmount * 5 / 100;
    _createDebt(a.user, a.txHash, debt);
}
```

Evidence: `BL-P02` and `BL-P03`.

8 Medium Findings

8.1 M-01 - `deposit()` Inflates Accounting Without Transferring Tokens

`IncentivesPool.deposit(amount, reason)` increments `poolBalance` and `totalReceived` without transferring tokens or checking the actual balance. A holder of `DEPOSITOR_ROLE` can create fictitious accounting and later distributions will fail.

Reproduction: grant the role to a wallet with no tokens, call `deposit(1_000 ether, ...)`, and compare `poolBalance()` against `actualBalance()`.

```
function deposit(uint256 amount, bytes32 reason) external nonReentrant {
    uint256 beforeBal = hackToken.balanceOf(address(this));
    hackToken.safeTransferFrom(msg.sender, address(this), amount);
    uint256 received = hackToken.balanceOf(address(this)) - beforeBal;
    require(received == amount, "fee-on-transfer not supported");
    poolBalance += received;
    totalReceived += received;
    emit Deposited(msg.sender, received, reason);
}
```

Evidence: `HC-M01`, 256-run Foundry fuzzing, and the Halmos counterexample.

8.2 M-02 - Immediate Fee Exemption and Stale Eligibility

The documentation requires 100,000 HACK staked **for** 12 months. The implementation enables the benefit as soon as the user commits to a one-year duration. The boolean may also remain active after the annual position is withdrawn when another shorter position keeps the aggregate above 100,000.

Reproduction 1: annual stake -> immediately call `activateNoCommission()`.
Reproduction 2: annual stake + monthly stake -> activate -> mature/withdraw annual stake -> exemption remains because of the aggregate.

Eligibility should be derived from a specific qualifying position, not a persistent boolean or total aggregate.

```
function hasNoCommission(address user) public view returns (bool) {
    uint256 id = qualifyingStakeId[user];
    Stake memory s = stakes[user][id];
    return s.active
        && s.amount >= 100_000 ether
        && block.timestamp >= s.startTime + 365 days;
}
```

Evidence: `HC-H03` and `BL-P04`.

8.3 M-03 - Reputation Claim Fails Across Epoch Rollover

`registerWinner()` stores the bonus under one epoch, while `claimBonus()` always looks up the epoch calculated at claim time. A winner registered near the end of an epoch loses a still-valid three-day window after rollover.

Recommended remediation: use an explicit `bonusId` or `cycleId` and validate `registeredAt + 3 days`.

```
function claimBonus(bytes32 bonusId) external {
    Bonus storage b = bonuses[bonusId];
    require(b.winner == msg.sender, "not winner");
    require(block.timestamp <= b.registeredAt + 3 days, "window closed");
    require(!b.claimed, "claimed");
    b.claimed = true;
    pool.distribute(msg.sender, b.amount, "reputation");
}
```

Evidence: `HC-M02`.

8.4 M-04 - Off-Chain Events Lack Unique IDs and Idempotency

Attendance, hiring, events, and several rewards are counters or direct payments without a consumable identifier. Backend retries or repeated signed messages can count the same fact more than once.

Reproduction: call `rewardTalentHired(educator, talent)` twice for the same hiring. Both calls pay because there is no `hiringId`.

```
mapping(bytes32 => bool) public processedAction;

function rewardHiring(bytes32 hiringId, address educator, address talent)
    external
    onlyRole(ENFORCER_ROLE)
{
    require(!processedAction[hiringId], "replay");
    processedAction[hiringId] = true;
    pool.distribute(educator, 5_000 ether, "educator_hiring_bonus");
}
```

Evidence: `HC-M04`.

8.5 M-05 - `Ownable` and `AccessControl` Administration Diverge

Transferring `owner()` does not transfer `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `BURNER_ROLE`, or `PAUSER_ROLE`. The former operator retains critical powers and the new owner may have no administrative authority.

Recommended remediation: use one administration model, preferably `AccessControlDefaultAdminRules`, a multisig administrator, and two-step acceptance. Verify that the previous account renounces roles after acceptance.

```
contract HackToken is ERC20, AccessControlDefaultAdminRules {
    constructor(address admin)
        ERC20("HackChain", "HACK")
        AccessControlDefaultAdminRules(2 days, admin)
    {}
}
```

Evidence: `HC-L01`.

8.6 M-06 - Recruiter Bonuses Do Not Prove Activity, Registration, or Coherent KYC

“Active for seven days” is implemented only as elapsed time since registration. KYC can be rewarded without requiring registration, and hiring records lack IDs.

Recommended remediation: a signed attestation should include recruiter, KYC provider, result, unique identifier, issue time, expiration, and nonce. The contract must require `isRegistered` and consume the ID once.

```
function claimKycBonus(KycAttestation calldata a, bytes calldata sig) external {
    require(isRegistered[a.recruiter], "not registered");
    require(a.recruiter == msg.sender, "wrong claimant");
    require(!usedKyc[a.kycId], "kyc used");
    require(_verifyKycSigner(a, sig), "invalid attestation");
    usedKyc[a.kycId] = true;
    pool.distribute(msg.sender, 200_000 ether, "kyc_bonus");
}
```

9 Low and Informational Findings

9.1 L-01 - Unsafe ERC-20 Transfer Handling

Several `transfer` and `transferFrom` calls either handle the returned boolean manually or do not use a consistent safe-transfer wrapper. Apply `SafeERC20` uniformly and validate that configured token dependencies contain contract code.

9.2 L-02 - Stake Index Is Validated Too Late

The array element is referenced before checking `stakeIndex < length`, causing a Solidity panic instead of the intended custom error. Check the bound first and access storage only after validation.

9.3 L-03 - Expired Advanced Membership Retains an Active Flag

`active` may remain `true` after expiration. This can block reactivation and may charge a cancellation penalty for a membership that has already expired. Derive activity from `expiresAt` or synchronize state before every action.

9.4 L-04 - Unbounded Loops and Batches

Stake searches and distributions over talent arrays may exceed the block gas limit as the system grows. Maintain aggregate indexes, limit batch sizes, and use pull-based or paginated distribution.

9.5 L-05 - Project Sponsor Can Be Overwritten and Has No Refund Path

A second project contribution overwrites `projectSponsor[projectId]` while both amounts remain accumulated. There is no deadline or cancellation path. Track contributions per sponsor and allow refunds of unspent funds after expiration.

9.6 I-01 - Thirty-Day Epochs Are Not Calendar Months

Mechanisms documented as monthly use `block.timestamp / 30 days`. Define an explicit `cycleId`, start and end timestamps, off-chain timezone policy, and claim window.

9.7 I-02 - Eligible Rewards Are Not Reserved

Eligibility does not reserve funds. A different module can consume the pool before a valid claimant submits the transaction. Create per-module and per-cycle budgets backed by the actual token balance.

9.8 I-03 - Deployment and Role Configuration Are Not Reproducible from the Received Repository

The received repository lacks idempotent deployment scripts, production addresses, CI configuration, and complete grant/revoke procedures. The audit package provides a reproduction environment, but the project must own a production-grade deployment pipeline.

9.9 I-04 - Code Quality and Observability

`ReputationBunises.sol` contains a filename typo, some state/events are unused, and critical dependency setters do not consistently emit events. Correct naming, remove dead state, and emit events for dependency and role changes.

9.10 L-06 - Emergency Pausing Is Not Granular and Can Block Withdrawals

Observation. HackToken has a global pause in `_update()`. This prevents new `transferFrom` operations into staking, memberships, or the incentives pool, but it also prevents `StakingContract` from returning principal through `transfer`. Fund-receiving modules do not inherit `Pausable` and do not offer an explicit withdrawal-only mode.

Risk. During an incident in staking or memberships, operators can either leave the complete system active or pause the token globally. The second choice reduces some attack surface but also freezes legitimate exits and compounds H-02.

Recommended remediation. Add per-module and per-flow pausing. Operations that create new exposure (`stake`, membership activation or renewal, new project funding, and administrative deposits) should use `whenNotPaused`. Principal withdrawals, safe cancellations, debt payments, and recovery functions should remain available unless the ERC-20 token itself is compromised. Assign the emergency role to a multisig and maintain events, a runbook, and pause/unpause tests.

```
contract StakingContract is AccessControl, Pausable, ReentrancyGuardTransient {
    bytes32 public constant EMERGENCY_ROLE = keccak256("EMERGENCY_ROLE");

    function pauseDeposits() external onlyRole(EMERGENCY_ROLE) {
        _pause();
    }

    function unpauseDeposits() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _unpause();
    }

    function stake(uint256 amount, uint256 duration)
        external
        whenNotPaused
        nonReentrant
    {
        _stake(msg.sender, amount, duration);
    }

    // Intentionally not guarded by whenNotPaused: principal remains withdrawable.
    function unstake(uint256 stakeId) external nonReentrant {
        _withdrawPrincipal(msg.sender, stakeId);
    }
}
```

Acceptance criterion. While deposits are paused, `stake()` must revert and `unstake()` must continue returning matured principal. The token-wide pause should be reserved for incidents in the ERC-20 itself.

9.11 I-05 - Public Mappings and Duplicate Getters

At least **16 exactly duplicated scalar access paths** were confirmed, in addition to struct wrappers. Examples include `apiIntegrationRewarded`/`hasApiIntegrationBonus`, `legacyCertsCount`/`getLegacyCertsCount`, `firstEventRewarded`/`hasFirstEventReward`, `profileBlocked`/`isProfileBlocked`, `isRegistered`/`getIsRegiste` `referredBy`/`getReferrer`, `totalStakedByUser`/`getTotalStaked`, and `degreeRewarded`/`hasDegreeReward`.

This does not create a direct exploit. It does increase deployment bytecode, duplicate ABI entries, enlarge the surface maintained by frontends and indexers, and expose storage representation as a public interface.

Professional recommendation. Select one strategy:

- For simple values that are intentionally part of the stable API, keep the mapping `public` and remove the exact wrapper.
- For derived state, structs, arrays, or data whose representation may change, use a `private` mapping with an underscore prefix and an explicit getter.

- For growing arrays, do not return the complete collection; expose count, indexed reads, and pagination.

```
mapping(address user => uint256 amount) private _totalStaked;

/// @notice Returns the active principal currently staked by `user`.
/// @param user Account whose active principal is queried.
/// @return amount Amount of HACK currently staked.
function totalStaked(address user) external view returns (uint256 amount) {
    return _totalStaked[user];
}
```

Removing or renaming getters changes the ABI and must be coordinated with the frontend, indexers, and tests.

9.12 I-06 - NatSpec Is Incomplete and Partially Misaligned with Behavior

The repository already includes NatSpec on most functions and therefore does not start from zero. However, a heuristic review of public and external declarations found minimally documented interfaces, many parameters without `@param`, and **no return values documented with `@return`**. Custom errors also omit context and useful values, while several public variables only have ordinary comments.

Some statements do not exactly match the implementation. For example, the no-commission NatSpec says "staked for at least 12 months," while the function only checks that a one-year duration was selected. Historical comments such as `CHANGE 3` and `CHANGE 4` describe editing history rather than the final specification.

Recommendation. Document the complete ABI with `@notice`, `@dev`, `@param`, and `@return`; explain roles, trust assumptions, pause behavior, checks-effects-interactions order, units, cycles, events, and errors; and state security invariants where behavior is not obvious. NatSpec must describe the current code, not NatSpec that does not match the current behavior.

```
/// @notice Withdraws a matured staking principal and records its reward as debt.
/// @dev Uses checks-effects-interactions.
///      Remains available while deposits are paused.
/// @param stakeId Index of the caller's staking position.
/// @return principal Amount of HACK returned to the caller.
/// @return rewardDebt Reward recorded for later collection from the incentives pool.
function unstake(uint256 stakeId)
    external
    nonReentrant
    returns (uint256 principal, uint256 rewardDebt)
{
    // ...
}
```

9.13 I-07 - Conditional Modernization to `ReentrancyGuardTransient`

The audited commit uses OpenZeppelin 5.0.2 and `ReentrancyGuard`. Current OpenZeppelin 5.x documentation provides `ReentrancyGuardTransient` for networks that support EIP-1153. The transient variant has been available since OpenZeppelin 5.1 and cannot be used on incompatible networks.

Polygon PoS PIP-31 includes EIP-1153 in its Cancun feature set. A separate compatibility build was performed with OpenZeppelin 5.6.1, Solidity 0.8.24, and `evmVersion=cancun`; the contracts compiled after replacing the import and base contract. This does not replace integration testing on the exact deployment network.

Decision. The migration is a modernization and potential gas optimization, not remediation for a demonstrated exploit: the existing `ReentrancyGuard` still provides reentrancy protection. Upgrade the dependency in a controlled change, pin the exact version, compile for Cancun, execute the complete test suite, measure bytecode/gas, and test on a fork or testnet of the target chain.

```
import {ReentrancyGuardTransient} from
    "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";

contract MembershipSystem is AccessControl, ReentrancyGuardTransient {
    // Existing nonReentrant modifiers remain in place.
}
```

Do not deploy bytecode containing TLOAD/TSTORE to a network without EIP-1153. Multichain deployments should maintain separate build profiles or artifacts.

9.14 Reviewed Criterion without a Finding - `Contract__Error()` Convention

The `Contract__Error()` convention is valid but is not a mandatory Solidity or OpenZeppelin standard. Errors declared inside a contract are already contextualized by that contract's ABI and can be referenced in tests as `StakingContract.InvalidDuration.selector`. OpenZeppelin generally uses semantic names such as `ERC20InsufficientBalance` or `AccessControlUnauthorizedAccount`, but does not require the double-underscore pattern.

A mass rename solely for style is therefore not recommended: it changes the ABI and requires updates to tests, frontends, and indexers without improving security. Improving semantics and adding useful parameters is recommended:

```
error InvalidAddress(address account);
error InsufficientPoolBalance(uint256 available, uint256 required);
error StakeNotMatured(uint256 unlockAt, uint256 currentTime);
```

A contract prefix may be adopted as an internal style rule when errors are declared at file scope, shared across modules, or mixed across ABIs, provided the project applies it consistently.

10 Business-Conformance Gaps

10.1 BL-01 - Allocation, Vesting, and Liquidity Lock Are Not Enforced On-Chain (High)

Documentation. Maximum supply of 1,000,000,000 HACK; allocation categories of 10/5/10/15/30/20/10%; advisors receive 1% from treasury; team, advisor, presale, and private-sale vesting; initial liquidity locked for at least one year.

Observation. `HackToken` enforces the global maximum supply, but a `MINTER_ROLE` can mint up to the cap to any address. No allocation vaults, vesting schedules, or LP-lock contracts are in scope.

Risk. A compromised or misconfigured minter can violate the publicly described tokenomics without exceeding the global cap.

Recommended remediation. Mint initial allocations directly to independent vaults (`VestingWallet` or tranche-based vesting), revoke the minter after the approved plan is completed, or restrict minting by category. Lock LP tokens or the liquidity-position NFT with a verifiable locker.

```
VestingWallet teamVault = new VestingWallet(
    teamMultisig,
    uint64(tge + 365 days),
    uint64(730 days)
);
hack.mint(address(teamVault), 100_000_000 ether);
```

The team must clarify whether "12-month lock plus 24-month release" means a 12-month cliff followed by 24 months of linear vesting. Boundary tests must reflect the approved interpretation.

10.2 BL-02 - The 5% Platform Transaction Fee Is Absent (High)

The business plan and mechanism specification state that platform transactions pay 5% in HACK to treasury unless the user qualifies for an exemption. No `CommissionSystem` or settlement contract exists and

no production flow consumes `hasNoCommission`.

Risk. A primary treasury revenue stream is missing, and the staking exemption has no verifiable economic effect.

Recommended remediation. Centralize settlement in a router that computes the fee, collects HACK, transfers the net amount, and checks exemption eligibility in the same state transition.

```
function settle(
    address payer,
    address payee,
    uint256 gross,
    bytes32 actionId
) external {
    require(!processed[actionId], "duplicate");
    processed[actionId] = true;

    uint256 fee = staking.hasNoCommission(payer) ? 0 : gross * 5 / 100;
    hack.safeTransferFrom(payer, treasury, fee);
    hack.safeTransferFrom(payer, payee, gross - fee);
    emit Settled(actionId, payer, payee, gross, fee);
}
```

Product must decide whether the fee is added to the price (`gross + fee`) or deducted from the recipient (`gross - fee`).

10.3 BL-03 - Personalized Classes Are Not Implemented (Medium)

The documented flow lets an educator set a price of at least 1,000 HACK per hour, a talent schedule and attend, and the educator receive payment. No equivalent flow exists.

Recommended remediation. Implement `LessonEscrow` with offer, price/duration, talent acceptance, deposit, bilateral confirmation, timeout, cancellation, and dispute handling. Do not pay the educator immediately at booking.

```
function book(bytes32 lessonId, address educator, uint256 amount) external {
    require(amount >= 1_000 ether, "minimum");
    hack.safeTransferFrom(msg.sender, address(this), amount);
    lessons[lessonId] = Lesson(msg.sender, educator, amount, Status.Funded);
}
```

10.4 BL-04 - Advanced Membership Is Incomplete (Medium)

Only the 50,000 HACK monthly level is implemented. The 140,000 quarterly and 550,000 annual levels are absent. Cancellation charges an additional 1,000 HACK but does not return an unused portion, while a business diagram includes a refund request.

Recommended remediation. Define tiers, proration/refund policy, and accounting for funds already sent to treasury/pool. A safer design keeps funds in escrow and releases them over time. If the approved policy is “no refund,” remove the refund path from every document.

10.5 BL-05 - Academic Membership Cancellation Is Missing and Sources Conflict (Medium)

The mechanism table defines pricing and the 50/50 split, while the business-plan diagram shows cancellation, a penalty, and a refund request. The contract has no academic-membership cancellation flow.

Required action. Approve one normative specification before implementation. The auditor should not infer a financial policy from contradictory documents.

10.6 BL-06 - Open-Source Project Funding Does Not Validate Roles, Repository, or Talents (Medium)

`fundProject` accepts any wallet, a zero `projectId`, and repeated contributions that overwrite the sponsor field. GitHub, open-source status, and active-talent membership exist only in comments.

Recommended remediation. Create a project registry containing a repository/commit attestation, permitted sponsor roles, a talent list or Merkle root, a deadline, and refund rules. Payouts should be pull-based.

10.7 BL-07 - Impersonation and Plagiarism Do Not Follow the Documented Process (Medium)

The impersonation penalty calls `_applyPenalty(..., blockProfile=false, ...)`, so it does not block the profile. The documents alternate between deleting the profile and blocking it until correction. For external plagiarism, tokens reach treasury but no atomic or mandatory payment to the affected party exists. `unblockProfile` is not tied to a `caseId`, correction, or payment.

Recommended remediation. Model explicit cases with states such as `Reported -> Confirmed -> DebtCreated -> Corrected/Paid -> Closed`, an evidence hash, and separate permissions for confirmation and closure.

10.8 BL-08 - Business Facts Rely on Trusted Off-Chain Calls without Attestations (Medium)

Attendance, 15 minutes in an interview, hiring, degree completion, activity, KYC, API integration, plagiarism, and employment are external facts. In the current design, an enforcer call is generally sufficient and frequently has no unique ID or source signature.

Recommended remediation. Use EIP-712 signatures, nonces, expiration, and an `actionId`. For complex facts, use an attestation registry and let reward modules consume each attestation once.

10.9 BL-09 - DAO/Governance Is Documented but Is Outside the Repository (Informational)

The whitepaper describes community governance and the presentation says staking enables DAO participation. No governance contract was included in scope. Because the roadmap presents it as future work, this is classified as a scope gap rather than a vulnerability in the received code.

Before launch, define vote snapshots, delegation, quorum, timelock, flash-loan-voting protection, and the parameters governance is allowed to change.

10.10 BL-10 - Documentary Contradictions and Regulatory Exposure (Medium)

The review identified, among others, these inconsistencies:

- Mass sale: one table specifies 5% **of the sale**, while the diagram shows 5% **of total token holdings**.
- Impersonation: one source says the profile is deleted; the diagram says it is blocked, corrected, and unblocked.
- Memberships: diagrams include refund/cancellation paths that the tables or contracts do not fully define.
- Business plan: treasury percentages and monthly returns to private investors for three years are described, while the disclaimer states that token holders have no economic rights or profit expectations.

Risk. Engineering cannot correctly implement an ambiguous normative source. A disclaimer may also fail to resolve the substance of economic claims made elsewhere; specialized legal review is required.

Required action. Produce a versioned specification approved by product, tokenomics, security, and legal counsel. Every mechanism should state its formula, actor, source/destination, snapshot time, states, cancellation rules, replay protection, and dispute policy.

11 Recommended Remediation Architecture

11.1 Layer 1 - Token and Treasury

- HackToken administered by a multisig and AccessControlDefaultAdminRules.
- Vesting vaults per allocation.
- Minter constrained or revoked after the token generation event.
- Separate treasury and incentive pool with module budgets.
- Verifiable LP lock.

11.2 Layer 2 - Business Evidence

- EIP-712 AttestationRegistry.
- Unique IDs for events, interviews, hires, KYC, certificates, views, and projects.
- Nonces, expiration, schema versioning, and revocation.
- Separate issuer roles per evidence type.

11.3 Layer 3 - Payments and Rewards

- CommissionRouter for settlement and platform fees.
- RewardBudget with reserves per mechanism and cycle.
- LessonEscrow and ProjectEscrow.
- Staking with principal separated from reward liquidity.
- Reputation with explicit cycles and snapshots.

11.4 Layer 4 - Penalties

- Case and debt system instead of opportunistic transferFrom enforcement.
- Snapshot or attestation of the violation and economic base.
- Profile blocking independent from token collection.
- Dispute, correction, payment, closure, and audit trail for each case.

12 Prioritized Remediation Plan

12.1 Blocking before a Public Testnet

1. Remediate H-01 through H-05.
2. Implement BL-01 and BL-02 or remove those promises from the launch scope.
3. Approve a formal penalty and membership specification.
4. Add IDs/attestations to paid off-chain actions.
5. Add reproducible deployment scripts and a role matrix.
6. Add granular inflow pausing without blocking principal withdrawals.

12.2 Before Mainnet

1. Resolve every Medium and Low finding.
2. Add budgets/reserves and solvency monitoring.
3. Obtain specialized legal review of private sales, returns, and public documents.
4. Execute backend and identity-provider integration tests.
5. Re-audit the final diff and simulate deployment on a fork/testnet.
6. Remove duplicate getters, complete NatSpec, and formally choose a ReentrancyGuard/ReentrancyGuardTran
   build profile for each network.

13 Minimum Post-Remediation Acceptance Tests

- A referral cannot be paid before 30 days and each referralId is consumed once.

- A staker recovers matured principal when the pool is empty, the module is paused, and the reward role is revoked.
- Self-views and duplicate views revert; snapshots do not change after a cycle closes.
- A violation creates a block/debt even if the user has zero allowance or later moves tokens.
- A mass-sale penalty uses pre-sale balance and a formally defined supply source.
- `poolBalance <= actualBalance` for every deposit/distribution sequence.
- Fee exemption exists only after 365 elapsed days for a specific active position.
- Reputation claims remain valid across a cycle boundary while inside their claim window.
- Duplicate events, hires, and KYC attestations revert.
- Administrative transfer removes privileges from the former account.
- Allocation and vesting sum exactly to 1,000,000,000 HACK.
- The 5% fee is correctly charged or waived in the actual settlement path.
- Membership cancellation/refund matches the approved specification.
- When deposits are paused, new inflows revert and principal withdrawals remain available.
- The Cancun build using `ReentrancyGuardTransient` is tested on an EIP-1153 fork/testnet; incompatible bytecode is never deployed to an older network.
- The ABI contains no duplicate scalar getters and every public/external item has complete, accurate NatSpec.

14 Package Reproduction Guide

```
unzip Hackchain_Smart_Contract_Audit_2026-07-11.zip
cd Hackchain_Smart_Contract_Audit_2026-07-11
npm ci
npm run compile
npm run test:js
npm run test:forge
npm run lint:forge
```

Relevant files:

- `test/security.spec.js`: 16 local PoCs.
- `foundry/test/SecurityPoC.t.sol`: Foundry tests.
- `foundry/test/HalmosPoolInvariant.t.sol`: symbolic property.
- `evidence/mocha-security.txt`: output of the 16 PoCs.
- `evidence/slither.*`: Slither execution and JSON.
- `evidence/halmos.*`: symbolic counterexample.
- `evidence/modernization/`: separate transient-guard compatibility build.
- `reports/contracts/`: contract-by-contract review.
- `agent-handoff/`: checklist and pending work for the implementation agent.

15 Limitations

An audit reduces risk; it does not prove the absolute absence of vulnerabilities. Results apply to the stated commit and dependencies. Any change to logic, roles, addresses, backend, tokenomics, or contracts requires diff review. Mechanisms based on real-world facts are only as trustworthy as the evidence provider, its keys, and its dispute process.

16 Final Conclusion

The code provides a useful modular foundation, but its current security does not match the level of trust promised by the business model. The immediate priority is not adding more rewards; it is preserving principal custody, making eligibility verifiable, preventing replays, defining snapshots, and ensuring that the central economic flows actually exist.

The audited system should not be deployed with real funds. Granular pausing, ABI cleanup, complete NatSpec, and a controlled transient-guard upgrade improve resilience and maintainability, but do not

replace remediation of High and Medium findings. After those findings and the normative specification are resolved, the final diff, deployment scripts, roles, tokenomics, and backend-attestation integration should be re-audited.

17 Appendix A - Evidence and Technical Environment

Item	Result
Commit	`290377e9862aeabaa2814bcd0bac9ddad2707ce8`
Solidity	`0.8.24`
OpenZeppelin in audited commit	`5.0.2`
OpenZeppelin modernization check	`5.6.1`
ReentrancyGuardTransient	Compiles with solc 0.8.24 + `evmVersion=cancun`; target-network test still required
Mocha/Ganache PoCs	16 passing
Foundry	7 passing; 256-run fuzz test
Slither	Successful execution; signals manually triaged
Halmos	Expected counterexample for `deposit()` solvency
Mythril	Installation not completed in this environment; recorded in handoff
Echidna	Not available; proposed properties recorded in handoff

18 Appendix B - Business-Document Hashes

Mecanismos del token HACK.pdf

`6cfbaf0982ed551b2e5b82cc49542a4d7cee0dba9ac3d7a4703dbf1b510ee124`

Plan de negocios.pdf

`5004babd0b771e28ba125200f8b156077426a15adfe8d62def4ab1d4c31ad856`

Whitepaper HackChain 1.1.0 (1).pdf

`1d08f647bbefd217a2c5bc190666d2b5876c1e8e69a17ef07a11aa704b5a8fba`

Diagramas de mecanismos.pdf

`bd63a573e4c9d3c345fbc39c162bf9107ba7ab845b505f8992bf33022bd8c4d0`

Conoce Hack Chain_20260508_183540_0000_compressed.pdf

`d341a58235f336d890dbec88fd15bd87952b414b21ab05abe9ca89982696b257`

19 Appendix C - Technical References

- Solidity Style Guide: naming conventions and project consistency.
- Solidity NatSpec Format: full documentation of the public ABI.
- OpenZeppelin Contracts 5.x Utilities: `ReentrancyGuard` and `ReentrancyGuardTransient` requirements.
- OpenZeppelin Contracts releases: v5.6.1 used for the isolated compatibility check.
- Polygon PIP-31: EIP-1153 in Polygon PoS Cancun features.
- Local evidence: `evidence/modernization/oz-5.6.1-reentrancyguardtransient-compile.txt`.

20 Appendix D - Severity Definitions

- **Critical:** immediate systemic loss/capture or complete control under realistic conditions.

- **High:** material loss, locked funds, or failure of a core economic mechanism under achievable conditions.
- **Medium:** exploitable non-conformance or material risk requiring privilege, timing, or integration conditions.
- **Low:** robustness, gas, compatibility, or secondary-logic issue with limited impact.
- **Informational:** technical debt, future scope, or operational recommendation.